LearnAble Study Notes

Breadth-First Search (BFS)

Note: No external sources were supplied for this material. Therefore, these notes are classified as general educational content, not research-verified content.

Study Notes: Breadth-First Search (BFS)

1. Topic Overview

Breadth-First Search (BFS) is a fundamental graph traversal algorithm used in computer science. It explores a graph (or tree) level by level, meaning it visits all the neighbors of a node before moving on to the next level of neighbors.

BFS is commonly used to find the shortest path in unweighted graphs, test if a graph is bipartite, and solve puzzles like mazes or social network connections (e.g., "degrees of separation").

2. Key Concepts and Definitions

- **Graph:** A data structure consisting of **vertices** (nodes) connected by **edges** (links). Graphs can be directed (one-way edges) or undirected (two-way edges).
- **Queue:** A linear data structure that follows the **FIFO** (First-In, First-Out) principle. BFS relies heavily on a queue to keep track of which node to visit next.
- **Visited Array/Set:** A tracking mechanism used to ensure that each node is visited only once. This prevents the algorithm from getting stuck in infinite loops (which can happen in graphs with cycles).
- **Level/Depth:** The distance (measured in the number of edges) from the starting node.

3. Detailed Explanations

How BFS Works (Step-by-Step)

1. **Choose a starting node** (often called the root or source). Mark it as visited and push it into a queue.
2. **While the queue is not empty:**

- Remove (dequeue) the front node from the queue. This is your current node.
- Process the current node (e.g., print its value, check if it's your target).
- Look at all the neighboring nodes of the current node.
- For each neighbor: If the neighbor has **not** been visited yet, mark it as visited and push it into the queue.

3. Repeat step 2 until the queue is completely empty.

Why use a Queue?

A queue ensures a **First-In, First-Out** order. Because nodes are added to the queue as we discover them level by level, the queue naturally forces the algorithm to process older discoveries before newer ones. This creates the "breadth-first" effect.

Complexity Analysis

- **Time Complexity:** $O(V + E)$, where V is the number of vertices and E is the number of edges. Every vertex and every edge is visited at least once.
- **Space Complexity:** $O(V)$. In the worst-case scenario (like a "star" graph where one central node connects to all other nodes), the queue will hold all vertices at the same time.

4. Examples

Visualizing BFS on a Simple Tree

Imagine a tree structure where Node A is at the top, connected to B and C. Node B is connected to D and E.

```
A

/ \

B C

/ \

D E
```

Tracing BFS starting at Node A:

1. Start at `A`. Mark `A` as visited. Queue = `[A]`
2. Dequeue `A`. Process `A`. Its unvisited neighbors are `B` and `C`. Mark them visited and enqueue them. Queue = `[B, C]`
3. Dequeue `B`. Process `B`. Its unvisited neighbors are `D` and `E`. Mark them visited and enqueue them. Queue = `[C, D, E]`
4. Dequeue `C`. Process `C`. It has no unvisited neighbors. Queue = `[D, E]`
5. Dequeue `D`. Process `D`. No unvisited neighbors. Queue = `[E]`
6. Dequeue `E`. Process `E`. No unvisited neighbors. Queue = `[]`

Final Traversal Order: `A -> B -> C -> D -> E`

5. Summary

- **Definition:** BFS is a level-order graph traversal algorithm.
- **Core Data Structure:** Queue (FIFO).
- **Key Advantage:** Guarantees the shortest path in an unweighted graph.
- **Tracking:** Always use a "visited" check to avoid infinite loops in cyclic graphs.
- **Performance:** Time complexity is $O(V + E)$; Space complexity is $O(V)$.

6. Five Practice Questions with Answers

Question 1

What data structure is primarily used to implement Breadth-First Search?

- **Answer:** A queue (following the FIFO principle).

Question 2

Why is a "visited" array or set necessary when running BFS on a graph?

- **Answer:** It prevents the algorithm from visiting the same node multiple times and keeps it from getting stuck in infinite loops if the graph contains cycles.

Question 3

What is the time complexity of BFS in terms of vertices (V) and edges (E)?

- **Answer:** $O(V + E)$, because every vertex and edge is processed.

Question 4

True or False: BFS explores as deep down into a branch as possible before backtracking.

- **Answer:** False. That describes Depth-First Search (DFS). BFS explores level by level, visiting all neighbors of a node before going deeper.

Question 5

Consider an undirected graph where you want to find the absolute minimum number of flights (edges) between City A and City B. Should you use BFS or DFS, and why?

- **Answer:** You should use **BFS**. BFS explores level by level, which guarantees that the first time it reaches the target node (City B), it has found the shortest path (fewest edges) in an unweighted graph.